R Programming

Suprit Saha
Tutorial Three

March 8, 2015

1 Factors

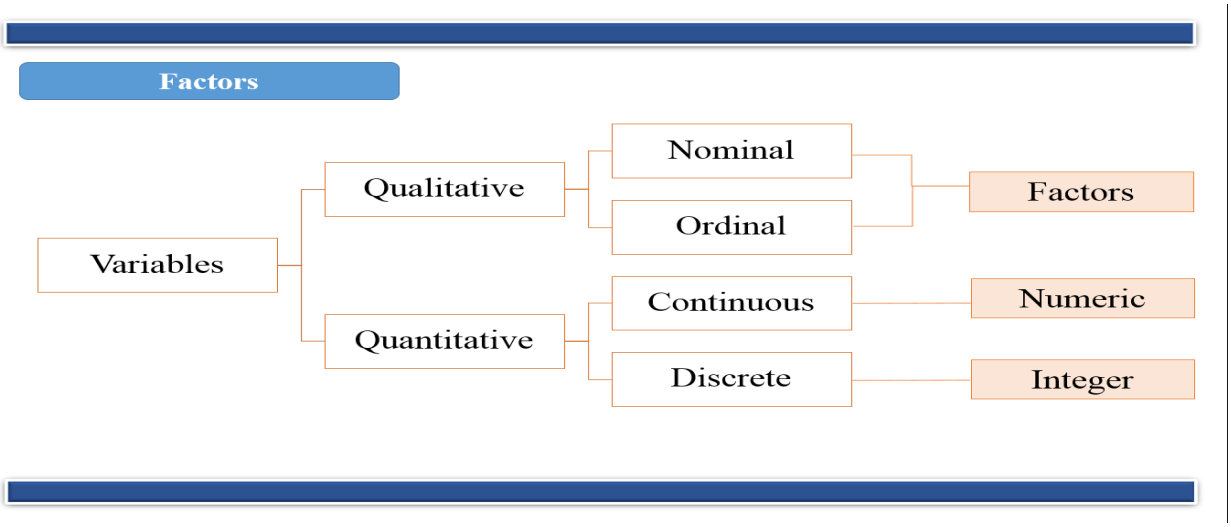

Figure 1: Variable and data types

Categorical (nominal) and **ordered categorical (ordinal)** variables in R are called **factors**. The function **factor()** stores the categorical values as a vector of integers internally. The factor function is used to create a factor

Nominal data: Factor

```
data <- c("Type1", "Type2", "Type1", "Type1","Type3")
fdata <- factor(data)
rdata <- factor(data, labels=c("I","II","III"))
```

Ordinal data: Ordered Factor

```
status <- c(Ailing", "Improved", "Excellent", Ailing")
status <- factor(status, order= TRUE, levels=c("Poor", "Improved", "Excellent"),
ordered=TRUE)
```

2 Data Structures Detailed

2.1 Vectors

- **Editing vectors**

```
x <- c(23, 12, 43, 1)
x <- c(x[1:3], 456, x[4]) #Insert 456 before 1
```

Note: Always reassign the vector while editing/appending the vector

- **Length of vector**

```
length(x)
```

Note: Length of a null vector is 0

- **Declaration**

```
y<- c(5,12,13)
#Alternative
y<- vector(length=3)
y[1] <- 5
y[2] <- 12
y[3] <- 13
```

Note: Do not need to mention the variable type of the vector like in other languages e.g. **C**

- **Vector Arithmetic**

```
x<-c(1,2,3)
y<-c(2,4,6)
x + y #addition
x * y #multiplication
y / x #division
x ^ y #exponentiation
y %% x #just the remainder
y %/% x #Integer part of the quotient
```

Note: R can be used as a calculator. Note that the operations occur element by element

- **Vector Indexing**

```
x <- c(1, 4, 5, 12, -9)
x[ c(1,3) ] #Extract 1st & 3rd elements of x
x[ 1:3 ] #Extract 1st , 2nd & 3rd elements of x
x[ -5 ] #Exclude the 5th element of x
x[ -c(2, 4) ] #Exclude 2nd & 4th element of x
```

Note: Frequently used operation. Negative subscripts mean that we want to exclude elements in our output

- **Generating vectors**

```
x<- c(2, 4, 6, 8, 10, 11, 11, 11, 11, 11.5, 11.5, 11.75, 11.75, 12, 13, 14, 15)
x1<- 12:15 # : Produces consecutive integers
x2<- seq (from=2, to=10, by=2) # seq generates a AP sequence
x2.1<-seq(from=2, to=10, length=5)
x3<- rep(11, times=4) #Repeat constants
x4<- rep(c(11.5, 11.75),each=2)
x<- c(x2, x3, x4, x1)
```

Note: Very useful in data creation. Frequently required

- **Using all() and any()**

```
x<-1:15
any(x>4)
all(x>4)
```

Note: any() & all() functions are shortcuts to report whether any or all of their arguments are TRUE

- **Vectorized Operations:**
  One of the most effective ways to achieve speed in R code is to use operations that are vectorized. Meaning that a function applied to a vector is actually applied individually to each element.

```
u<- c(5, 2, 8)
v<- c(1, 3, 9)
u>v #Checking condition element wise
sum(u > v) #Number of instances where condition is TRUE
sqrt (1:9) # Mathematical functions are vectorized
w<- c(1.2, 3.9, 0.4)
round(w)
u+4
```

Note: In R, vectorization is often done for both (a) readability and (b) performance

- **NA & NULL**

```
x<-c(12, NA, 20, 28)
mean(x)
mean(x,na.rm=TRUE)
y<-c(12, NULL, 20, 28)
mean(y)
```

Note: Missing data in R is represented by NA. NULL on the other hand represents non existent values

- **Vectorized ifelse() function:**
  In addition to the usual if-then-else construct found in most languages, R includes a vectorized version, the ifelse() function

```
x<-2:10
ifelse(x%%2==0, Even, Odd)
ifelse(x>5, x+2, x+3)
```

Note: Advantage over standard if-else construct is that it is vectorized, thus potentially much faster

- **Vector Element Names**

```
x<-c(1,2,4)
names(x)
names(x)<- c("a","b","c") #Assigning names
names(x)
names(x)<-NULL #Removing names
```

Note: Assign vector element names using names() function. Remove names by assigning NULL

2.2 Matrices

A matrix is a vector with two additional attributes: **number of rows** & **number of columns**. Matrices are special cases of arrays which can be multidimensional.

- **Creating Matrices**

```
y<- matrix(c(1,2,3,4), nrow=2, ncol=2)
#Alternative
y<- matrix(nrow=2,ncol=2)
y[1,1]<-1
y[2,1]<-2
y[1,2]<-3
y[2,2]<-4
y1<-matrix(c(1,2,3,4),nrow=2,ncol=2,byrow=TRUE)
#Alternative
y2<- cbind(c(1,2),c(3,4))
y3<- rbind(c(1,3),c(2,4))
```

Note: When we construct a matrix directly with data elements, the matrix content is filled along the column orientation by default.The byrow argument enables input to be read along row orientation form.**cbind()** & **rbind()** combines vectors, matrices, data frames by column & row respectively.

- **Matrix Computations**

```
mat1<-matrix(1:6,nrow=2,ncol=3)
mat2<-matrix(1:6,nrow=2,ncol=3,byrow=TRUE)
t(mat1) # Transpose
mat1+mat2 # Addition
mat1-mat2 # Subtraction
mat3<-mat1 %*% t(mat2) # Matrix multiplication
det(mat3) # Determinant
solve(mat3) # Inverse
mat1*2 # Element wise multiplication
mat1*mat2
mat1/2 # Element wise division
mat1/mat2
```

Note: Note the difference in symbols used for matrix multiplication & element wise multiplication

- **Matrix Indexing**

```
mat<- matrix(1:12,3,4)
mat[2:3, ] # Extract 2nd to 3rd row
mat[ , c(1,3)] # Extract 1st & 3rd column
mat[ ,-4] # Exclude 4th column
```

- **Matrix Indexing**

```
mat<-matrix(c(1,2,2,3,3,4),nrow=3,ncol=2,byrow=TRUE)
i<-mat[ ,2]>=3
mat[i, ]
# Alternative
mat[mat[,2]>=3, ]

z<-c(5,12,13)
x[z%%2==1, ]

m<-matrix(1:6,2,2)
m[m[ ,1]>1 & m[ ,2]>5, ]
m[m[ ,1]>1 & m[ ,2]>5, ,drop=FALSE]

m<- matrix(c(5,2,9,-1,10,11),3,2)
which(m>2)
```

Note: Matrix filtering is similar to vector filtering. Filtering criterion can be based on a different variable from the one to which filtering will be applied.The drop=FALSE argument retains the two dimensional nature of the data

- **Using the apply() function**

  One of the most famous & popular features is the **apply()** family of functions, such as **apply(), lapply(), sapply(), mapply(), tapply()**. Let us look at apply() which is used to evaluate a function over the margins of a matrix.

  apply(m, dimcode, f, fargs) where

  - m is the matrix
  - dimcode is the dimension, 1 if function applies to rows, 2 for columns
  - f is the function to be applied
  - fargs is an optional set of arguments to be supplied to f

```
m<-matrix(1:6, 3, 2)
apply(m, 2, mean) #Column means
apply(m, 1, max) #Row maximum
x<-matrix(rnorm(200), 20, 10)
apply(x, 1, quantile, probs=c(0.25, 0.75))#1st & 3rd quartile of each row
```

  Note: Do not loop , use apply() in R.Makes code compact, easier to read, modify and avoid possible bugs.

- **Adding/Deleting matrix rows & columns**

```
mat<- matrix(1:12, 3, 4)
one<- c(1, 1, 1)
mat<-cbind(one, mat)
odd<- c(3, 5, 7, 9, 11)
mat<-rbind(mat, odd)
mat<-mat[c(1,3), ]
```

  Note: cbind() & rbind() functions let you add columns & rows to a matrix

- **Naming matrices/Other functions**

```
mat<-matrix(1:4,2,2)
colnames(mat)
colnames(mat)<-c("a", "b")
rownames(mat)<-c("Row1", "Row2")
mat
dim(mat) #Get dimensions of matrix
nrow(mat) #Alt: dim(mat)[1]
ncol(mat) #Alt: dim(mat)[2]
```